

Lec 36 Practical Exercise - 1

Lec 36 Practical Exercise - 1

Lecture	36 / 63 + intro
Duration	33:04
Captions	English (auto-generated)
Words	4,771
Video	WTPywahiL9w
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all. Welcome to the hands-on session on fundamentals of generative AI and large language models theory and practice. In today's hands-on session, we'll be implementing the basic generative adversarial networks, in short called as GANs. So, we'll be seeing the implementation of a very basic architecture of GAN that is a vanilla GAN, which acts as a fundamental framework for development of more diffusion models or maybe DCGAN, StyleGAN and CycleGAN.

[00:33] So, as we have seen in the theory sessions, generative adversarial networks is a combination of two neural networks, that is a generator and a discriminator. Generator is the one which creates realistic samples or maybe we can call it as a fake samples and tries to match it to the realistic samples. And the discriminator acts as a classifier, which has to discriminate or predict the real images and the fake images. So, vanilla architecture or a vanilla GAN is a basic architecture which was introduced in the year 2014 and it is a combination of several fully connected networks. So, in today's implementation, we will see the vanilla GAN. So, let us start the hands-on.

[01:20] So, initial initial portion is the importing of the necessary libraries. So, importing the TensorFlow as TF, which is a deep learning framework in order to develop

the networks. From TensorFlow Keras, we import the layers in order to have uh several layers like input layers, the output layers and a combination of hidden layers. Then, we also import the numerical Python as np, which is required for dealing with arrays. And importing the matplotlib.pyplot as plt, which is required for data visualization.

[01:54] So, in our hands-on, we make use of the Fashion MNIST data set, which comprises of uh fashion uh images, including clothing and some of the accessories, such as uh t-shirts, trouser, pullover, dress, uh sandals, bags, and so on. So, it comprises of 10 different classes, and class zero will be t-shirts or tops. Class one is a trouser. Class two is pullover. Class three is dress. Class four is uh labeled as coat. Class five is sandal. Class six is shirt. Seven is sneaker. Eight is bag. And the ninth one is ankle boots. So, totally zero to nine classes, which comprises of 10 different classes in the Fashion MNIST data set.

[02:41] So, let us load this Fashion MNIST data set, and let us perform some kind of pre-processing on the data set. So, from TensorFlow Keras data set, which is a benchmarking data set, we make use of the Fashion MNIST data set, which is a benchmarking data set, and load it into a variable called as X_train. So, we have two sets of data here. One is the training and the test set. That is the conventional method of having two different uh samples or two different sets o- on the data set. But, in the case of GANs, which is required for uh generating the samples, we need only the training samples. We do not even uh need the labels. That's why we don't have this uh uh Y labels, that is the target variable.

[03:28] And we also do not make use of X_test, because it is completely uh unsupervised to form of learning. That's why we don't need labels, as well as the test data set. So, in case of testing the data set, uh the generator generates some fake samples, and the discriminator has to uh classify the real ones. That is, the ones which we have loaded uh from the Fashion MNIST data set, and the ones which is generated by the generator. So, there is no provision of uh using the X test samples. That's why we completely avoid X test, Y train, and Y test. And that is represented using the underscore here.

[04:06] Now, coming to normalizing to minus one and one. So, normalization is required in order to get the data samples or values in between certain range. In our previous hands-on, we have seen as to getting the values in the range zero to one. But here in this hands-on, we make use of the values as minus one to one. That is basically because we use make use of the tan h activation function. We have seen in our previous sessions that tan h gets the values in the range minus one to one. So, in order to have these values between the range minus one to one, we have this kind of arrangement here.

[04:44] X_train is equal to X_train as type float minus 127.5 divided by 127.5. That

means we are converting this training sample into floating data type and then subtract 127.5 and then divide by 127.5. So, let us see how exactly this works. We know that the tan h activation function gives the values between the range minus one to one. But the MNIST data set what we have taken is a gray scale set of gray scale images and the pixel values vary from zero to 255. So, we have pixel intensities varying from zero to 255.

[05:32] So, this has to be brought down to minus one to one. So, what if the we take the minimum value? And what if we take the maximum value? So, let us take the minimum value. So, X_{train} , which is zero, minus 127.5 divided by 127.5. So, this will give us the value as minus one, which which is the normalized value. And we have taken the minimum value. Now, let us take the maximum value. So, maximum value what we have in the pixel is 255 minus 127.5 divided by 127.5.

[06:14] So, on the numerator we get 127.5 divided by 127.5, which is equal to one. That's why now we have the values in the range minus one to one. This is the reason why we take this term minus 127.5. So, once we are converting into floating data type and doing this initial pre-processing, we then flatten the image so that the initial values, that is 28 cross 28 pixels, are now converted into a vector of the size 784. So, we are completely flattening the images.

[06:53] So, now when we print the training shape using shape function, training shape comprises of 60,000 samples and the number of variables are 784 in number. Now, coming to defining some of the hyper parameters. So, generative adversarial learning or networks is basically a kind of learning wherein the generator takes you takes in the random noise, maybe called as Z , or you can call it as a latent vector Z , and converts into some sample images. That is why we need to mention the dimensions or the random noise dimensions. That's why we have this latent dimension. So, latent dimension, for simplicity, we take 100.

[07:40] So, we have generating we are generating a noise vector of value 100. And let us divide the straining samples, which are 60,000 in number, into 128 batch size. So, that the first value, that is the first sample up to the 128th sample, that forms the first batch. Similarly, 129 to 256, that forms the second batch. Likewise, the number of batches continue in the training samples. And let the number of epochs be considered as 20.

[08:11] So, this is the initialization of certain parameters, hyper-parameters. Now, coming to defining the first part of the GAN, that is the generator. So, in order to define the generator, we make use of the sequential layers, that is having the layers start one over the other. So, that's why we make use of the function called as sequential. So, once we have the sequential in the layers, we take the input layer, and the input is based on the latent dimension. So, our input is of the size 100 now.

[08:42] From from this input vector, we have to get the fake images, and that is the

role of the generator here. And then, we make use of 256 neurons and the leaky ReLU activation function. Why we avoid the ReLU activation function is we have seen that ReLU activation function take the maximum value of zero and the value, that is zero and X. So, if X is negative, then the value returned is If X is negative, then the value returned will be zero. If X is positive, then the same value X is been returned.

[09:29] So, with this, what happens is some of the neurons will become inactive, or we are taking away the functionality of certain neurons. In order to avoid this, we make use of a new activation function called as leaky ReLU. So, it works in this way. It takes into account the maximum value of x comma a into x or we can call it as α into x , where a or α is a scaling value or a scalar. So, when the value is negative, say when x is negative, and let some value for a , which is maybe 0.2.

[10:10] Uh when x is negative, what happens in this case? Maximum of say minus five, 0.2 into minus five. So, this will be maximum of minus five comma minus one. So, the value returned will be the maximum value, which is minus one. Now, what if x is positive? Say five into 0.2 times five. So, this will be maximum of five comma one, which gives us the original value. So, with the help of this leaky ReLU function, we have a small leaking value, which makes the neuron still active, and it is not deactivated from the functioning. So, that's the function of leaky leaky ReLU.

[11:06] So, with the help of leaky ReLU and the scaling factor that is 0.2, we are able to get the dense layer. And in the next dense layer or the fully connected layer, we take the five 12 neurons and the same leaky ReLU. And coming to the last dense layer, which is comprising of 1024 uh neurons with a leaky ReLU value as uh 0.2, and the final or the output layer comprising of 784 because the input variables were 784 in number. That's why dense layers with 784 neurons and the activation function is tan h. So, what happens? We are able to generate the values in the range -1 to 1. That is why we need this tan h activation function.

[11:50] So, when we generate the summary of this generator, the different layers with the output shape and the number of parameters or trainable parameters looks like this. So, we have the first dense layer with the output shape as 256 and the number of trainable parameters are 25,856. So, how do we get this value is input shape is equal to latent dimension. Input shape is equal to latent dimension, which we have taken 100 in number.

[12:26] And then we have 256 neurons. 256 neurons. So, this will be input into number of hidden neurons plus the bias in the hidden neurons. So, input is equal to 100 variables 256 plus 256. So, this is going to be the number of trainable parameters, which which is 25,856. 856. So, we are also considering this bias into consideration. So, likewise when we consider the leaky relu, there is no trainable parameters. That's why it is zero.

[13:18] Uh same way next dense layer come with the output shape as 512. The same

way we have to calculate. So, 256 plus two Sorry, 256 into 2 512 plus 512. So, that is going to give us 131,584 trainable parameters. So, likewise when we calculate for each of the dense layers or the fully connected layers, we get the different trainable parameters. And when we sum it up, we get this huge uh you know number of trainable parameters. So, this is the summary of the generator.

[13:51] Next, we have the discriminator, which is acting like a classifier and classifies the fake images created by the generator and the ones which are the real images which is present in the Fashion MNIST dataset. So, how do we develop this discriminator is once again the sequential function. And we are stacking layers one over the other. So, first input shape will be equal to 784 because we have 784 input shape, which is defined earlier.

[14:20] Then, we take the opposite of what we have taken in case of the generator. So, 1024 neurons come first, followed by 512, 256. So, it is just opposite of the generator. Whatever variables we have taken or neurons we have taken in case of the generator. 1024, and then we have 512, and then 256. And at last, we have the dense layer. And the number of neurons will be equal to one because we have to we need only one neuron because it is acting like a binary classifier and giving us only one class, whether it is fake image or the real image.

[14:59] And the activation function is sigmoid because we have we have to get the probabilistic value, which lies between the range 0 to 1. So, whenever the value is greater than 0.5, then the class is one. Whenever the probability value is less than 0.5, the class will be zero. So, it can be considered as zero class as fake and the class one as the real image. So, when coming to the discriminator summary, the functionality is the same.

[15:27] The number of parameters calculation will remain the same. So, it is number of the input neurons or the input shape into the hidden neurons plus the bias associated with the hidden neurons. So, when we calculate, we get this particular value. So, we can look into one particular example here. Input shape is 784. 784 into 1024 plus 1024 will give us 803,840 parameters. So, when we sum up all these parameters, we'll be getting 40,000 6 1,460,225 different parameters.

[16:07] So, now we have defined the generator, that is the first neural network followed by the discriminator, which is the second neural network. Now, creating the GAN class. So, the first line, class GAN from the TensorFlow Keras model, we get this GAN class. And then we define the generator as well as the discriminator. So, [snorts] what is happening is we are first defining the GAN class, which creates a custom GAN model containing the generator, the discriminator, and the training logic.

[16:37] Then we take the constructor that receives both the generator and the discriminator and stores the value or stores them. And then we initialize the base Keras model using the super function here. And then to store the networks, that is the gen-

erator and the discriminator, we make use of these two lines. So, now the GAN object has access to both the networks, that is the generator as well as the discriminator. So, once this is defined, we have to make a track of the loss. Loss pertaining both both to the discriminator as well as the generator.

[17:15] So, gen- discriminator or initially at the very first epoch if we consider, the generator is very weak at generating the new samples. So, what happens is in the initial state the discriminator is very powerful and the loss associated with discriminator will be very less when compared to the generator loss. Now, when we continue with the adversarial learning, that is learning of the latent vectors, converting those learning latent vectors into realistic image, what happens is the generator becomes very powerful. And what happens to the discriminator loss? The discriminator loss slightly increases.

[17:59] So, we need to have a track of both the generator as well as the discriminator loss. That's why we need the trackers, which will track or update or store the values. So, from the metrics, we make use of the mean function and at every epoch we have we calculate the discriminator loss as well as the generator loss. So, we take the average value just to avoid too much deviation or just to have No, avoiding the outliers.

[18:31] Next coming to defining the metrics. So, as mentioned earlier, the metrics is the discriminator loss and the generator loss. So, we initially configure the discriminator optimizer, the generator optimizer. We can use any optimizer. For simplicity, we make use of the Adam optimizer here. So, Adam optimizer makes use of two estimates. One is the mean estimate, other one is the variance estimate. So, let us see the functionality of this Adam optimizer.

[19:20] » So, using this Adam optimizer, we'll be able to calculate the new weights, which is equal to the old weight minus eta into mean by or mean estimate plus the epsilon value. So, M_{cap} and V_{cap} refers to the mean estimate and the variance estimate. So, what is this mean estimate? This will be equal to β_1 into M plus one minus β_1 into GT , which is the gradient. And the variance estimate will be β_2 into V plus one minus β_2 into GT^2 .

[20:06] So, β_1 and β_2 are small values or uh scalar values, which has a control over the moving average of the gradients and the square of the gradients or we can call it as a past gradients and this is the squared gradients. So, when we substitute this M_{cap} and V_{cap} in the equation for the weights, we get the new weights based on the old weights. And this eta refers to the learning rate of the model. So, we have uh configured the two different optimizer. One optimizer or Adam one optimizer related to the discriminator and the second Adam optimizer related to the generator. So, using this, we make use of the loss function or we calculate the loss, back propagate the loss, back propagate the

[20:56] gradients and that is how the uh updation of the weights take place. So, this is all about configuring the optimizers. Now coming to the training step. How training happens in each step or each of the epochs. So, what we have is the set of real images, which is nothing but the Fashion MNIST data set images. We have made use of the batch size from the real images. That is the Fashion MNIST data set, we are converting into different batches of size 128.

[21:31] And then uh training the discriminator. So, discriminator will make use of the normal distribution of data or a randomly generated normal distribution or a Gaussian noise. And the shape will be same as that of the batch size as well as the latent dimension. So, in our case, the size will be 128 cross 100. And what is the generated images? It is the one which is generated by the generator, which makes use of the random latent vectors of size 100, and training will be equal to true. That means generator is made to train on this latent vector, which is a noise.

[22:11] And then we are combining the images. So, what we are concatenating or what combination of images we want is the real images what we have from the data set Fashion MNIST data set and one which is generated by the generator. So, in axis zero, that is we put one below the other. So, initially we have the real images, which has come from the uh Fashion MNIST data set. And then we have the fake images, which is generated by the generator.

[22:50] So, if we take the real images of size 128 and then 128, like this we have different batch size and goes up to 60,000 training samples. Then below that, we have the fake images. So, they are put or concatenated in a row. That's why we have axis is equal to zero here. That's why we have this axis is equal to zero. So, once we do this, we also need the labels. That is, let us label the real images as ones and the fake images that is created by the generator as zeros.

[23:33] So, from TensorFlow, we take ones as well as zeros and let the label come once again one below the other. That is, for the first training samples, that is, the real images, we have the labels as one, which indicates the true data. Followed by, after these 60,000 samples, we have the labels as zero for the fake images. So, once again, it is concatenated in the rows. And then, uh we have the labels. The labels are also created from the uniform distribution. Why uniform distribution is, we want the labels to be equally probable. That's why we have this uniform distribution, unlike the Gaussian distribution. And that is multiplied by a small uh scalar, which is of the value 0.05.

[24:24] So, let us create such labels. And now, we have the predictions, which is done by the discriminator. So, discriminator, for the discriminator, we give the combination of images, which comprises of the real image and the fake images. Training is equal to true. So, training takes place on the combined images. And then, the discriminator loss or a loss function, usually a bi- a binary cross entropy loss function, we can take.

[24:51] So, that is based on what whatever labels we have taken and whatever predictions done by the discriminator. So, it is going to compare the labels what we have created. That is created here. And whatever predictions done by the discriminator. So, based on the discriminator loss, we calculate the gradients. So, after the gradients are calculated, we have the updation of the weights. And then we create or we consider all such loss. That's why we have to create a zip file or a zipping of various values of the gradients and the trainable weights. So, whichever is the best possible weight that can be used for the next epoch.

[25:37] Now, coming to training the generator. So, generator takes into account the normal distribution or a Gaussian noise of the size batch size and latent dimension. So, 128 cross 100. And there should be some misleading label labels. That is it will consider ones. And the size will be batch size comma one here. So, from the generated images, which is a combination of latent vectors and the training happening, we make some prediction. So, prediction of the discriminator on the generated images when the training is happening.

[26:15] So, now what is this generator loss? So, whatever misleading labels that we have created for the fake images and whatever prediction done. If they are not equal, the number of class misclassifications will be more. So, with the help of this generator loss, we update the gradients, we update the weights. That happens in the back propagation. So, collectively, we make use of all the gradient values or trainable weights and then store it somewhere.

[26:46] So, where do we store it? Is using a tracker. Tracker is like a running calculator updates the states. So in our case updating of the state means updating of the discriminator loss and updating of the generator loss. So finally we return the discriminator loss and the generator loss. So this is how we have uh created the GAN class. Now coming to creation of the model, the GAN model which makes use of the generator, which makes use of the discriminator. And while doing the configuration, we make use of the optimizer. Already the importance of this Adam optimizer has been told. So learning rate is required and the beta values are also required.

[27:35] Usually we take beta value as .5 for generative adversarial networks because let us not have too much dependency on the past data. Suppose we increase or decrease the values, the dependency on the past data will be too much. Due to which the unrealistic images uh generation could be more. That's why to have a neutral value and to have less oscillations uh between the images, we can have beta value as .5. So uh finally we calculate the binary cross entropy loss. So binary cross entropy makes into uh binary cross entropy considers the true labels and the predicted labels. So whenever they are uh different from each other, the loss will be high.

[28:19] So now coming to training the GAN, that is fitting the GAN on the training data set. Number of epochs already defined earlier, 20 and batch size is equal to 128.

So when we run for 20 different epochs, we'll be able to see the discriminator loss as well as the generator loss. So discriminator loss .71 and generator loss is .89. So what happens is the discriminator is having less loss that means it is able to classify between the fake images created by the generator and the real images which were provided by the data set.

[28:57] Now coming to plotting of these losses, the discriminator and the generator loss. So let us make use of the figure function, fixed size is the argument, 8 in in width and 5 in in height. So let us plot the history of the discriminator losses. Similarly, we have a history or we have already tracked the generator losses. So let the label be discriminator loss, let the label for this be generator loss. And X label being epoch and Y label the set of losses.

[29:29] Title will be vanilla GAN training. So let us show the legend. So whichever colors which colors suits which color which particular loss. So blue one indicates the discriminator loss and the orange one indicates the generator loss. So we see that during the initial epochs, that is from zero to maybe seven epochs, what has happened is the discriminator loss is very less when compared to the generator loss. So this is what I mentioned earlier.

[29:59] In the initial stage, the generator will not be able to have a lot of learning on the images. So it is not able to get the realistic images, that's why the loss is more when compared to the discriminator. Then abruptly the discriminator loss further reduces, means it is having a good control over classification. And gradually after 10 epochs, what happens is the discriminator and the generator loss are moving together, they are hand in hand. So that's why we were able to see a very small difference between the generator and the discriminator loss, that is 0.71 and 0.89.

[30:40] So now let us generate some fashion images. Number of images is equal to 16. We may be able to take any number of images. So normal distribution of the data from which we get the random noise generated and the shape will be number of images that is 16 and the latent dimension. So 16, 100. So generator has to predict the noise. That is from this Z or the latent vector, it has to generate the images. So whatever images generated, let us reshape into the number of images that is 16 and 28 cross 28 pixel.

[31:18] So now let us have the generated image one after the other. So we are going to have from the zeroth image that is the zeroth index up to the 15th index. So that's why every iteration we show this plus one. So now let us show the generated images displaying. So for I in the range number of images that is 16, let us have a subplot of four rows and four columns and in every location let us have one after the other 16 different images. So we are trying to show the images that are generated by the generator. The color map is gray because a fashion MNIST is a combination of grayscale images.

[31:59] There is no access required so therefore it is off and the super title given to all these set of 16 images is generated fashion MNIST images. So let us show the images. So this is how the generated images looks like. So 16 different images we have generated. Some of these images we are able to you know find that as we have a set of shirt here. We have the sneakers or the shoes, the sandals, t-shirts, shirts. And hopefully this is a bag. So we are able to get some of the clothing images as well as the accessories.

[32:42] So, this is all about the implementation of vanilla GAN. So, vanilla GAN acts as the fundamental uh deep learning framework in order to generate the deep convolutional GANs or cycle GANs or style GANs. Thank you.